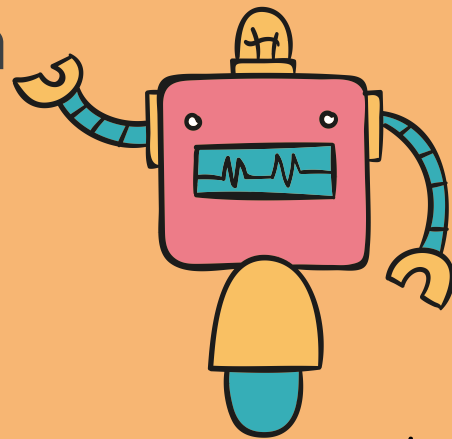
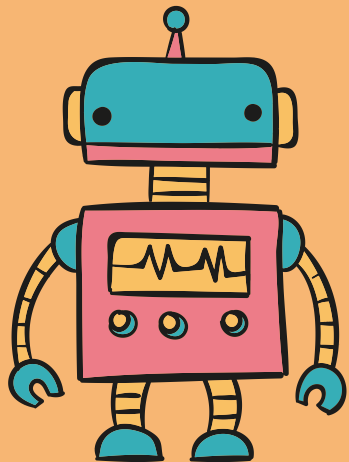


LLM Chatbot Battle

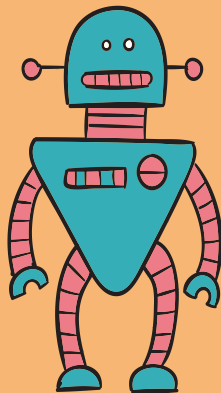
User Preference Prediction

Jiamin Yao, Yan Pang, Hangyu Yao
Deep Learning - 2025 Fall

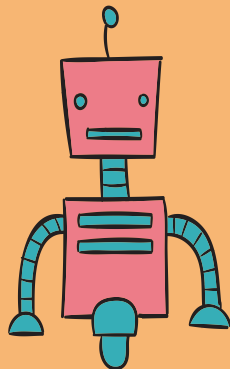


Team Member-Y3

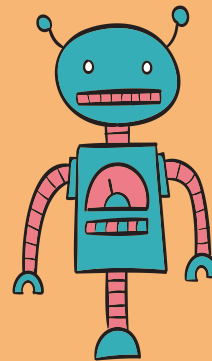
+



Hangyu Yao



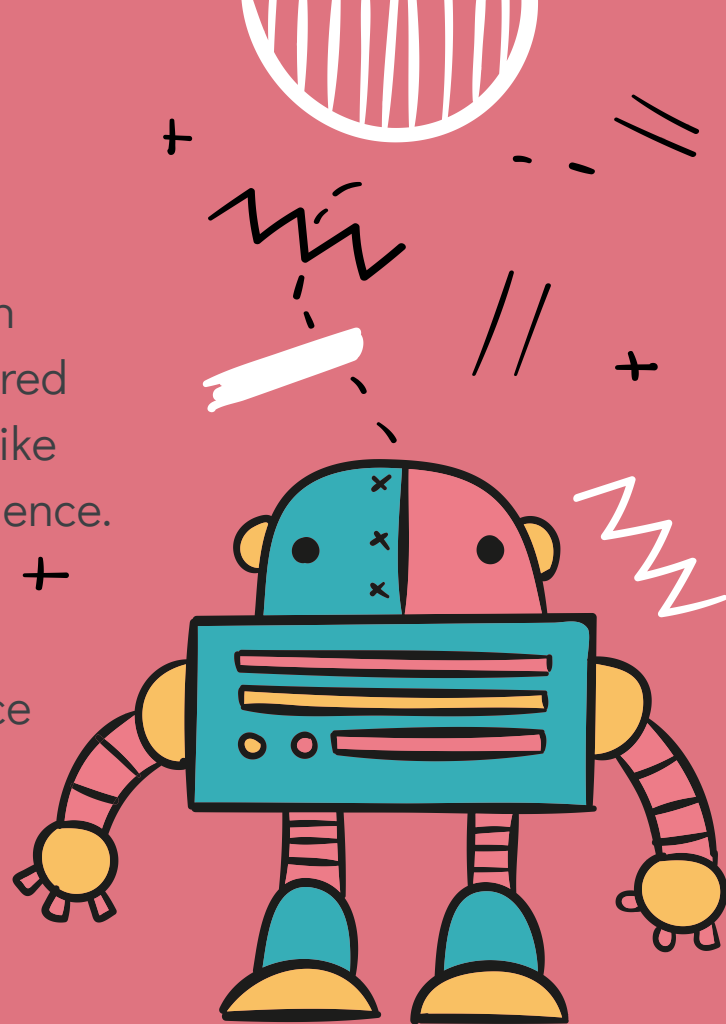
Jiamin Yao



Yan Pang

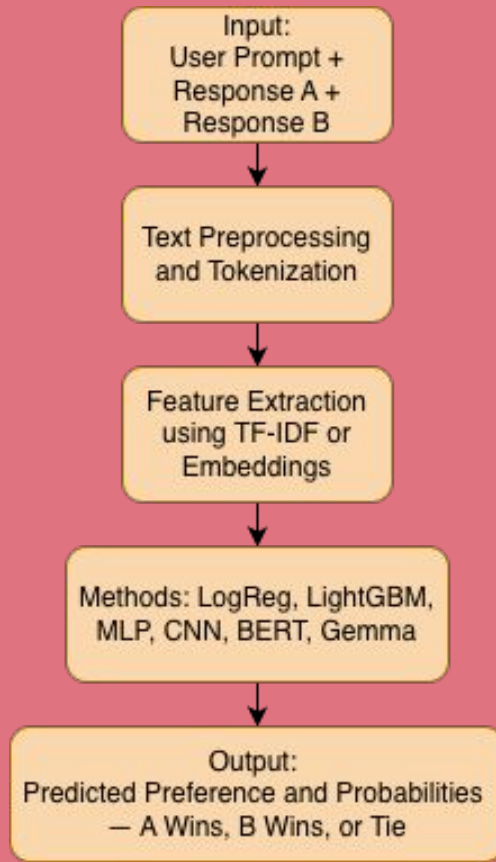
⚡ Background

Large language models (LLMs) are widely used in chatbots, but not all responses are equally preferred by users. Predicting which responses users will like can help improve chatbot quality and user experience. Previous work has explored traditional machine + learning, deep learning, and transformer-based methods for datasets with fixed human preference labels.



Motivation

Our project with with fixed human preference label focuses on comparing classical machine learning, deep learning, and transformer-based models. We will experiment with different parameters and fine-tuning strategies to identify the best approach for predicting user preferences between large language model responses.



Related Work

Category	Key Idea / Method
Latent Preference Learning ^[1]	Models user edits to infer hidden preferences and improve LLM alignment.
Conversational Recommendation ^[2]	Combines knowledge graphs with LLMs for personalized conversational suggestions.
Optimization-based Preference Modeling ^[3]	Uses Bayesian optimization to iteratively learn user utility functions from language-based feedback.
Tool-Augmented Reasoning ^[4]	Employs LLMs with external tools for reasoning-based, adaptive recommendations.
Synthetic User Simulation ^[5]	Builds LLM-based simulators to emulate user behaviors for large-scale preference data generation.

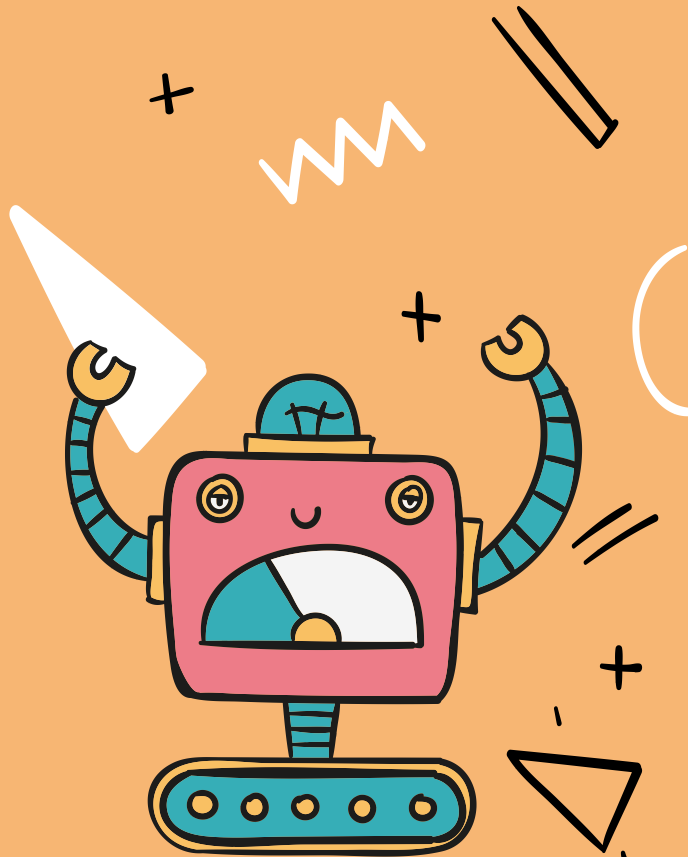
Our work complements these efforts by shifting focus from synthetic or domain-specific feedback to real human preference judgments from the Chatbot Arena dataset.

Instead of building user simulators or optimizing preference functions, we benchmark multiple models – from classical ML to LLM-based – to evaluate how effectively they can predict genuine user response preferences, providing a practical baseline for future preference-learning research.

Dataset

The dataset^[6] comes from the ChatBot Arena, where users give prompts to two LLMs and choose which response they like best. The goal is to predict which response a user will prefer and how likely a response will be selected as the winner. It includes:

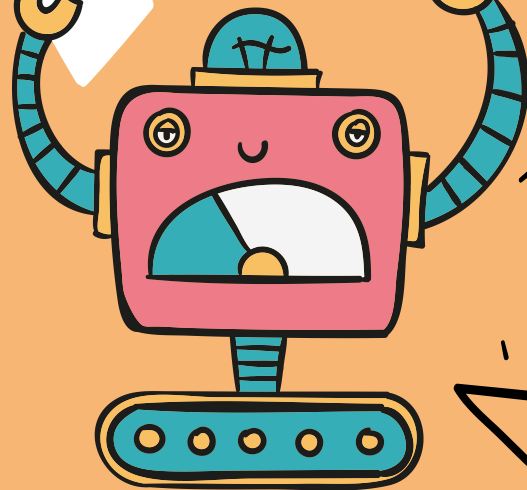
- train.csv – labeled comparison data (80% for training, 20% for validation)
- test.csv – unlabeled data for prediction submission, and
- example_submission.csv – a format guide for competition entry.



Dataset Features

Feature	Description
Id	A unique identifier for the row.
model_a / model_b	The identity of model A or B. Included in train.csv but not in test.csv.
prompt	The input prompt given to both models.
response_a / response_b	The response from model A or B to the given prompt.
winner_model_a / winner_model_b / winner_tie	Binary columns indicating the judge's preference. These are the ground truth target columns.

Shape: (57477, 9)



Methods⁺

Classical Machine Learning:

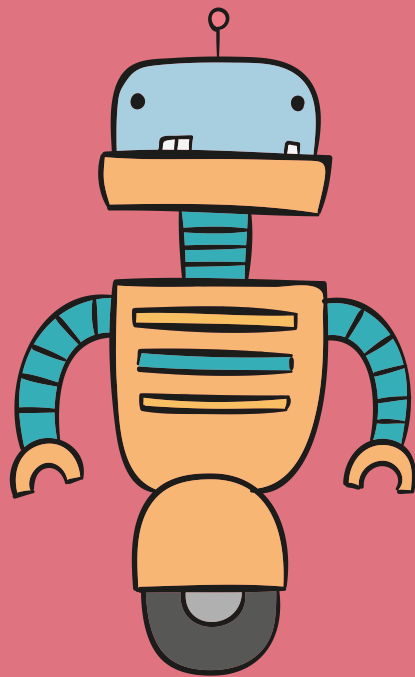
Logistic Regression^[7], LightGBM, etc

Neural Networks:

Feedforward Neural Networks (MLP)^[8],
Convolutional Neural Networks (CNNs), etc

Transformer:

BERT^[9], Gemma, etc



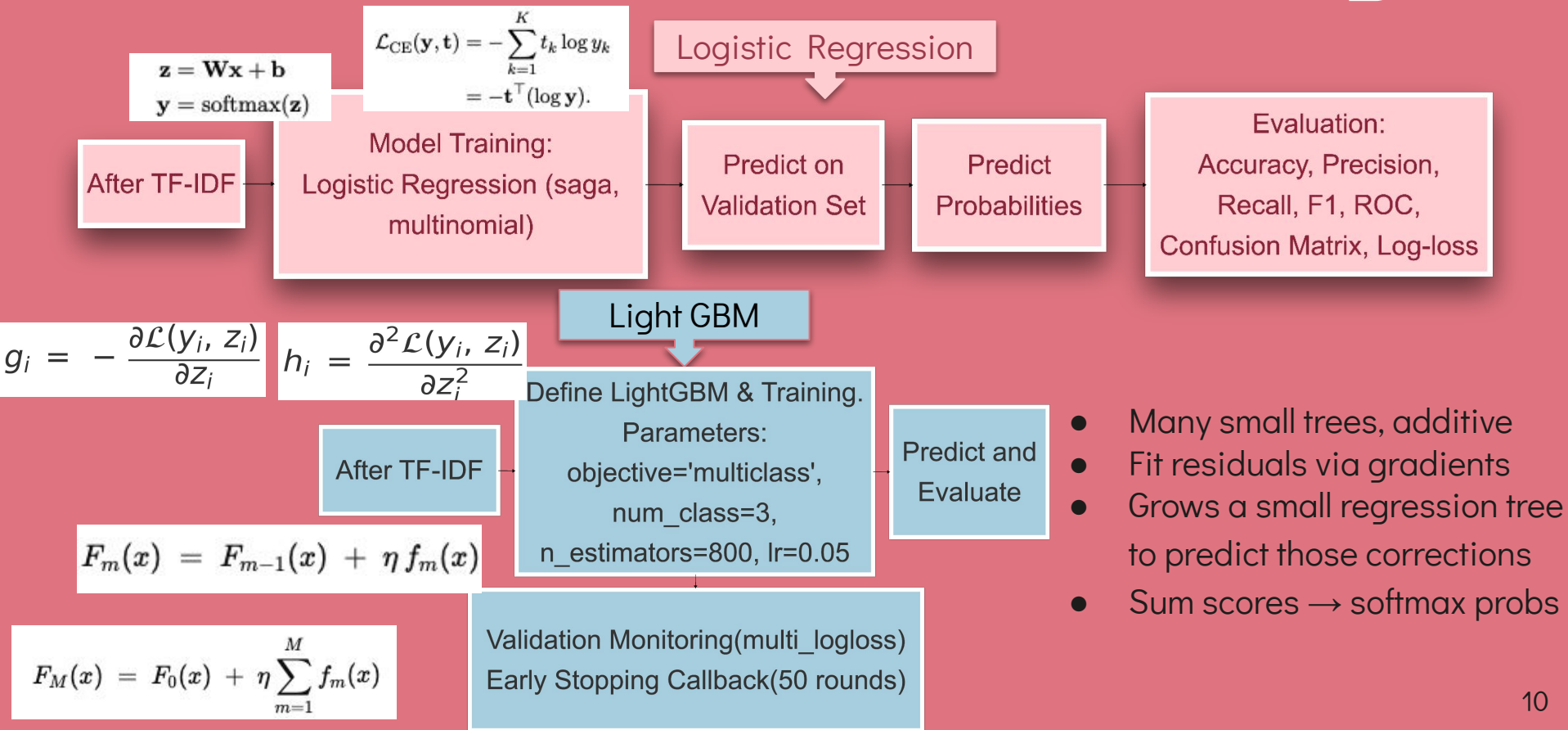
Data Processing

Label Mapping: Use the index of the max value in one-hot (A=0, B=1, Tie=2).

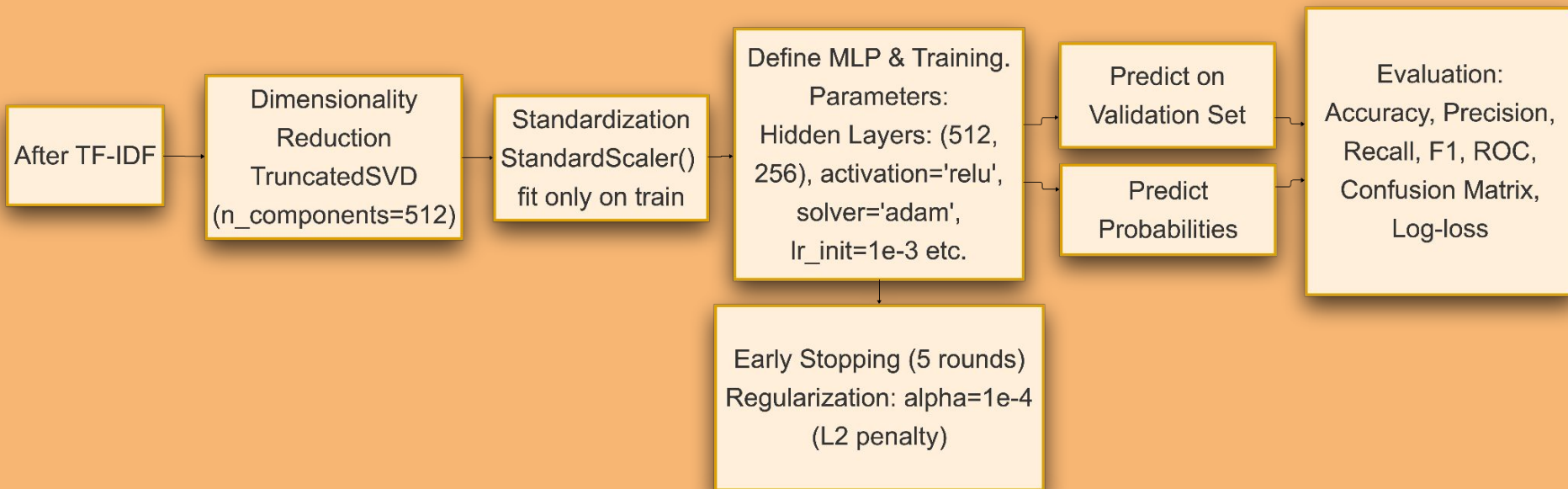
Model	Text Building	Feature Processing	Split Train/Valid
BERT (text-based)	prompt [SEP] response_a [SEP] response_b	Convert combined text → list	80/20
Gemma (instruction)	Instruction format ##prompts: ... \n, ##responses: ...	Convert combined text → list	80/20
TF-IDF Models	Use columns independently (prompt / A / B)	TF-IDF → hstack sparse matrix	80/20
Dual-Conv CNNs	Conversation sequences conv_a/b	Build pair sequences	80/20

$$\text{tfidf}(w, d) = \text{tf}(w, d) \cdot \log \frac{N}{\text{df}(w)}$$

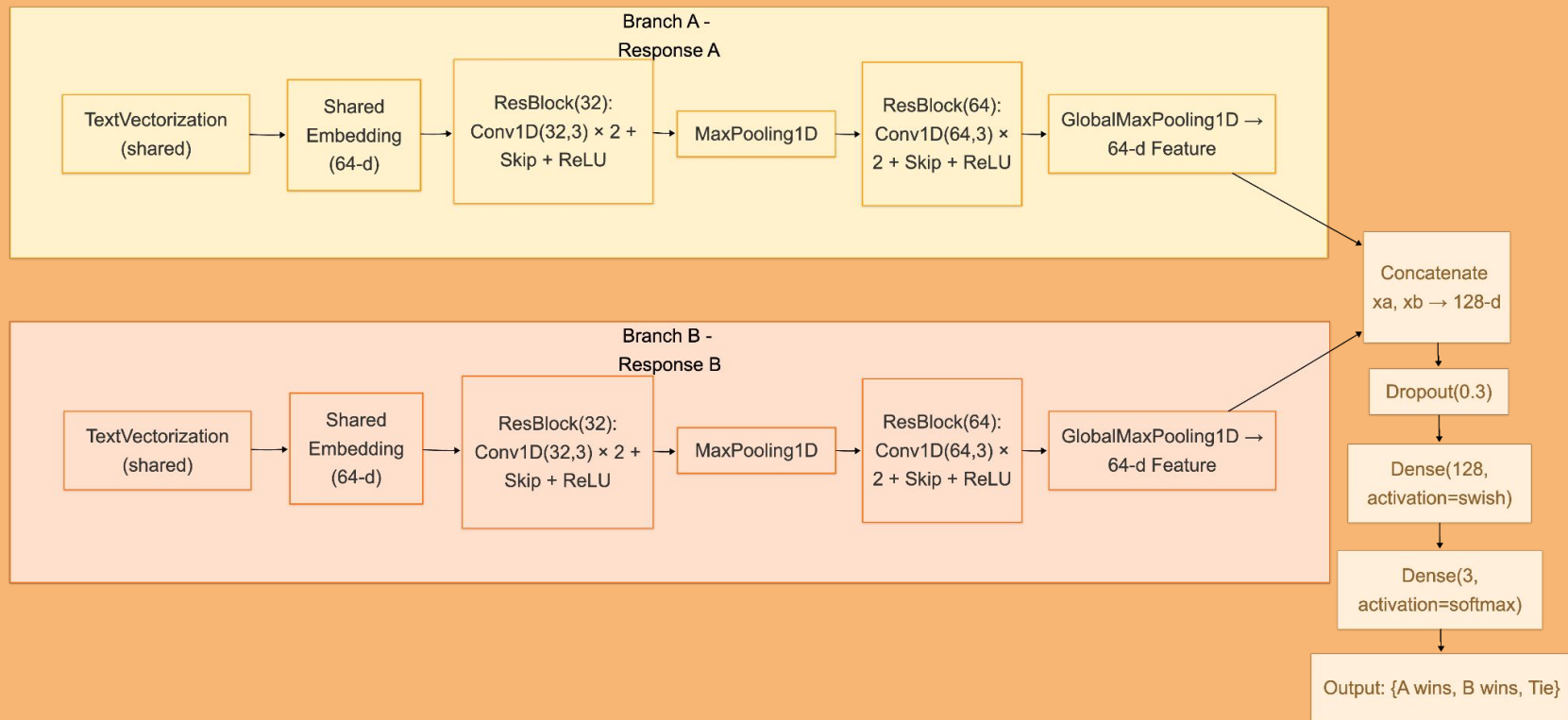
Classical Machine Learning



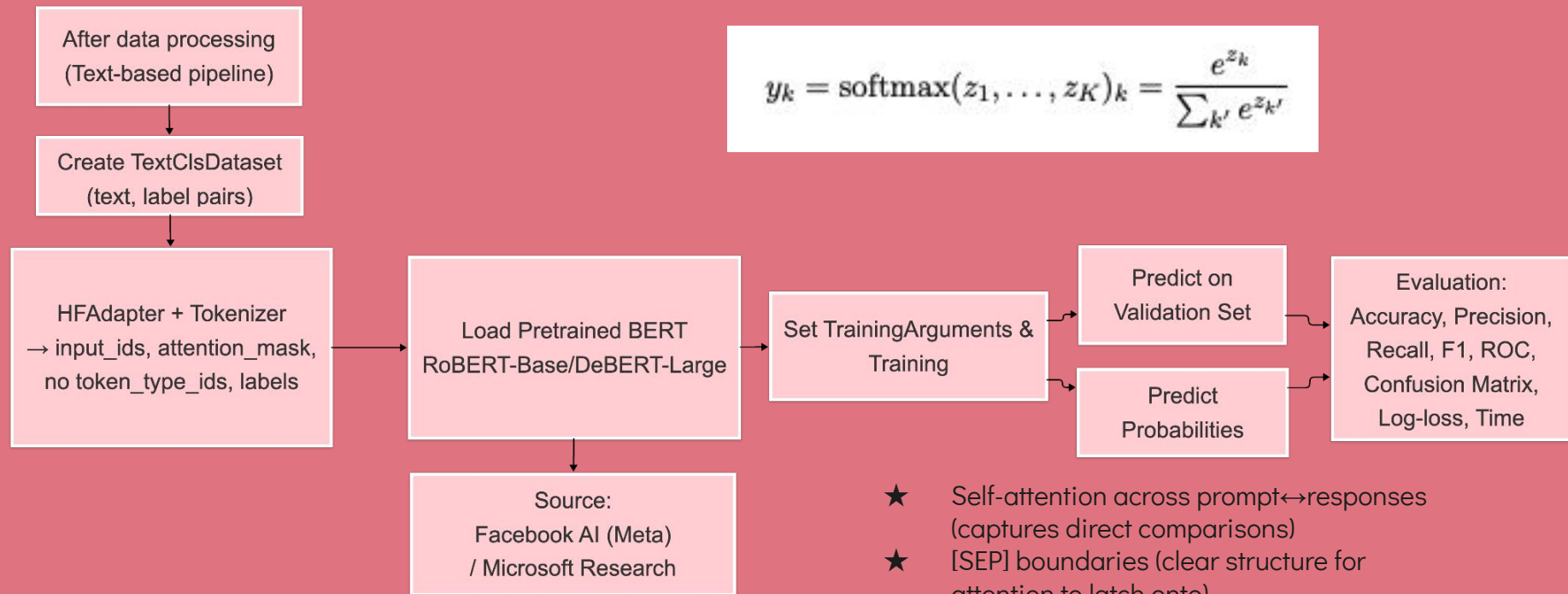
Neural Networks (MLP)



Neural Networks (CNN)



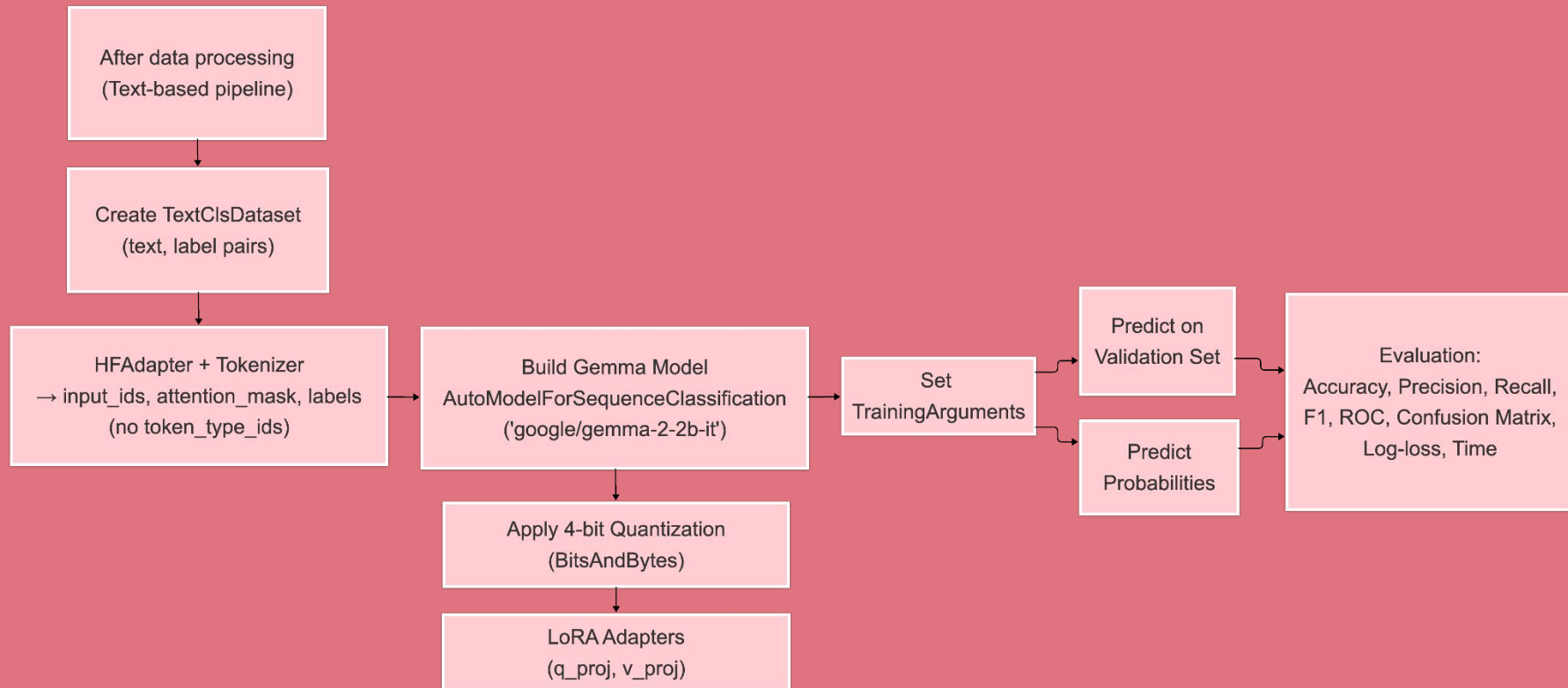
Transformer (BERT)



$$y_k = \text{softmax}(z_1, \dots, z_K)_k = \frac{e^{z_k}}{\sum_{k'} e^{z_{k'}}}$$

- ★ Self-attention across prompt↔responses (captures direct comparisons)
- ★ [SEP] boundaries (clear structure for attention to latch onto)
- ★ [CLS] summary (final signal the head reads)
- ★ Pretraining + small LR (strong prior, stable adaptation)

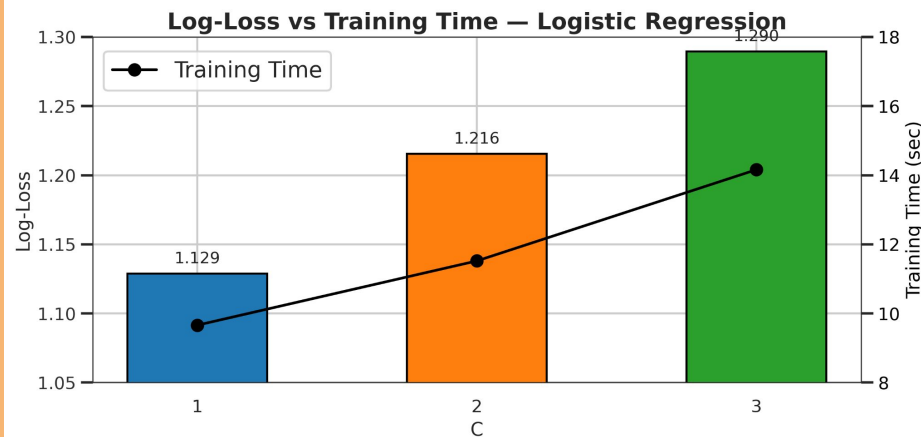
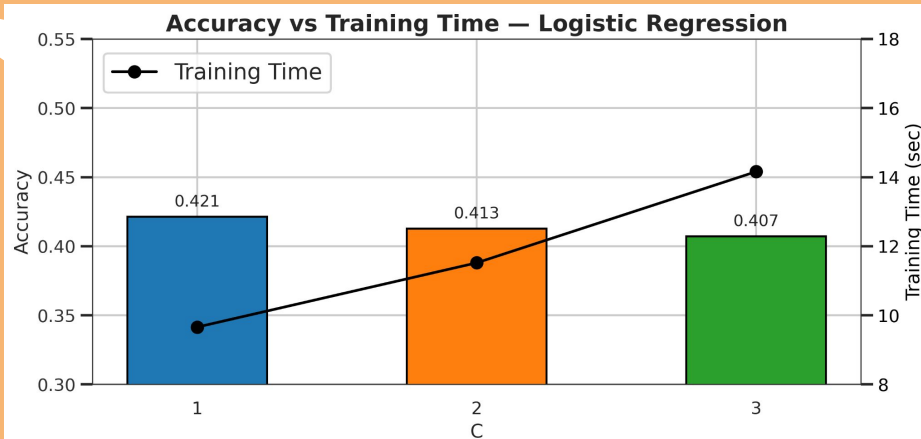
Transformer (Gemma)



Experiments Design

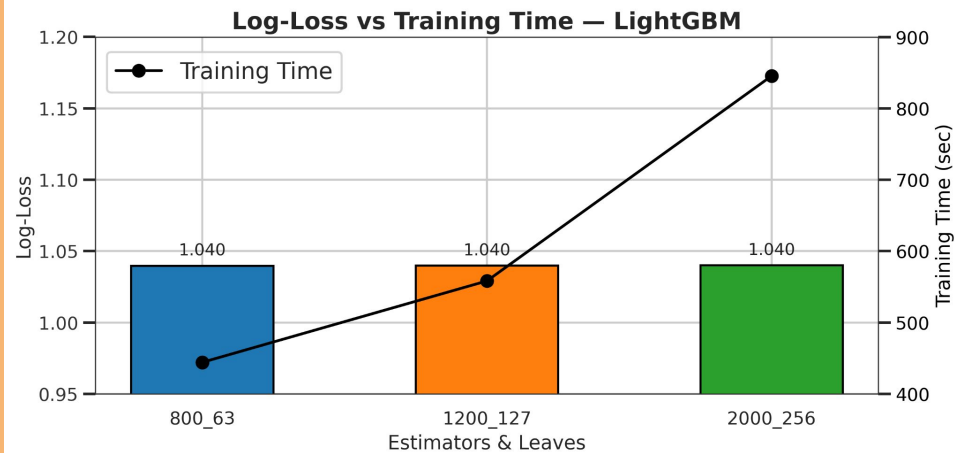
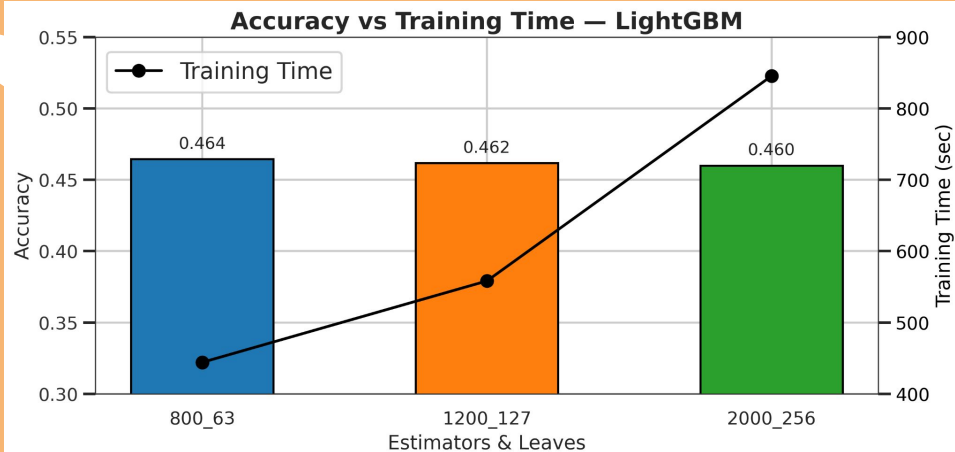
Category	Features / Data Aspects	Planned Experiments	Evaluation Strategy
Classical ML	- TF-IDF features (sparse numeric vectors) - Response length & token counts - Prompt-response similarity (cosine)	Train Logistic Regression/LightGBM on TF-IDF + simple numeric features	❖ Accuracy : Overall proportion of correct predictions. ❖ Precision : Fraction of predicted positives that are actually correct (averaged across classes).
Neural Networks	- Pretrained embeddings (BERT/SBERT sentence vectors) - Response length & token counts - Prompt-response similarity	Train MLP/CNN on embeddings, compare with classical ML baselines	❖ Recall : Fraction of actual positives that are correctly identified (averaged across classes). ❖ F1 score : Harmonic mean of precision and recall; balances both metrics.
Transformer s	- Raw prompt text + responses (input directly to Transformer) - Implicit embeddings learned during fine-tuning	Fine-tune BERT/Gemma for 3-class preference prediction (A wins, B wins, Tie)	❖ Confusion matrix : Visual breakdown of predictions vs. true labels, showing per-class errors. ❖ ROC-AUC : Measures how well the model distinguishes between classes across thresholds (One-vs-Rest for multiclass). ❖ Log-loss : Penalizes confident but wrong predictions; lower values indicate better calibrated probabilities. ❖ Training Time : Measure how many time it takes to train with provided data.

Exp Results - LogReg



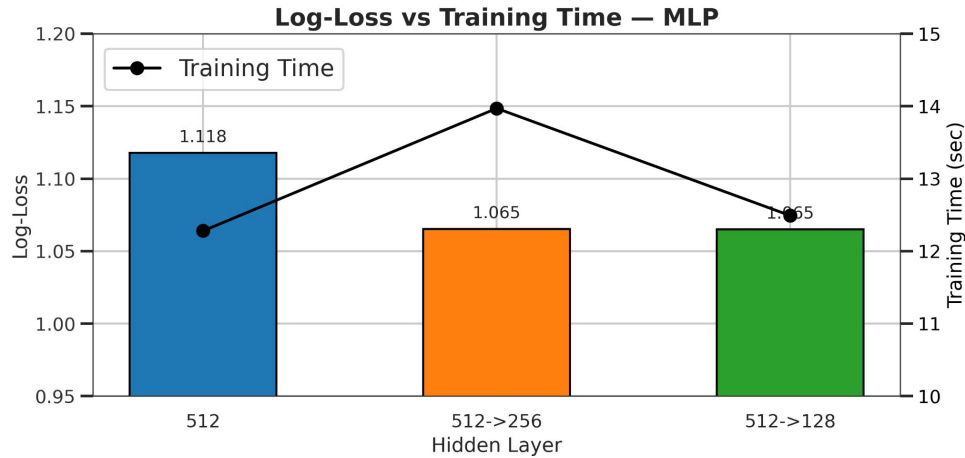
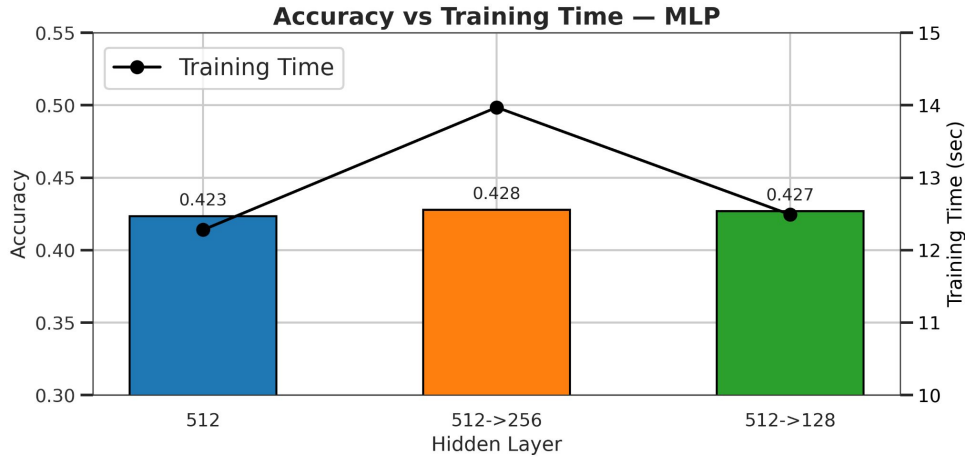
Method	C
Logistic Regression	1
Logistic Regression	2
Logistic Regression	3

Exp Results - LightGBM



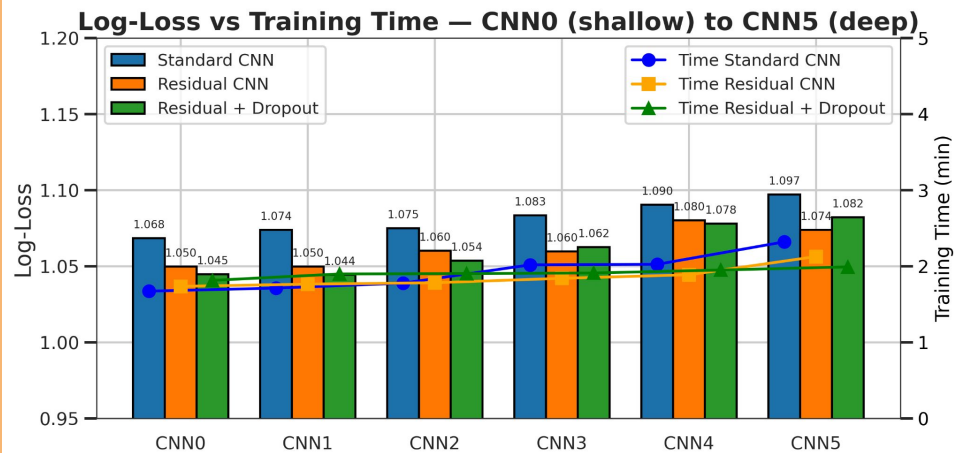
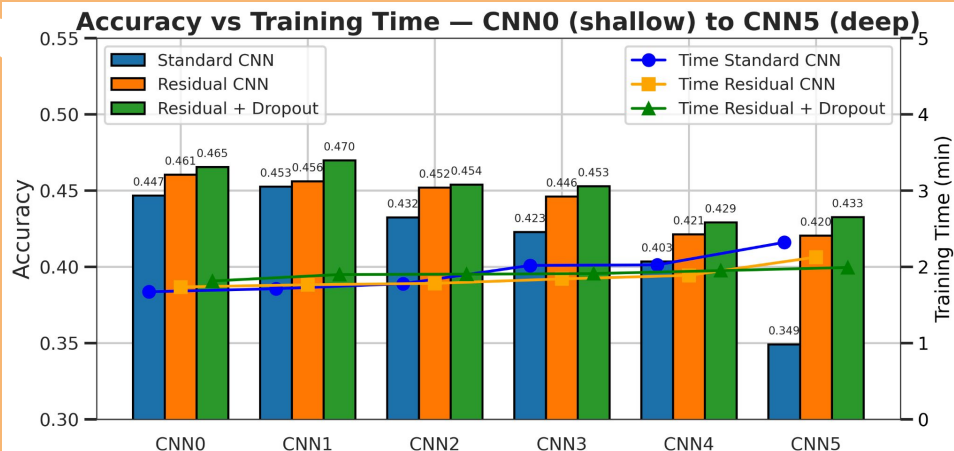
Moethod	n_estimators	num_leaves
LightGBM	800	63
LightGBM	1200	127
LightGBM	2000	256

Exp Results - MLP



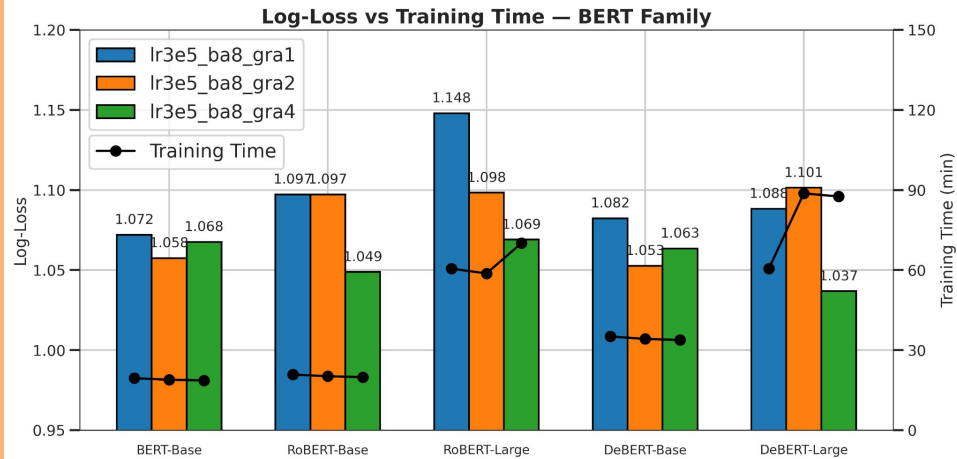
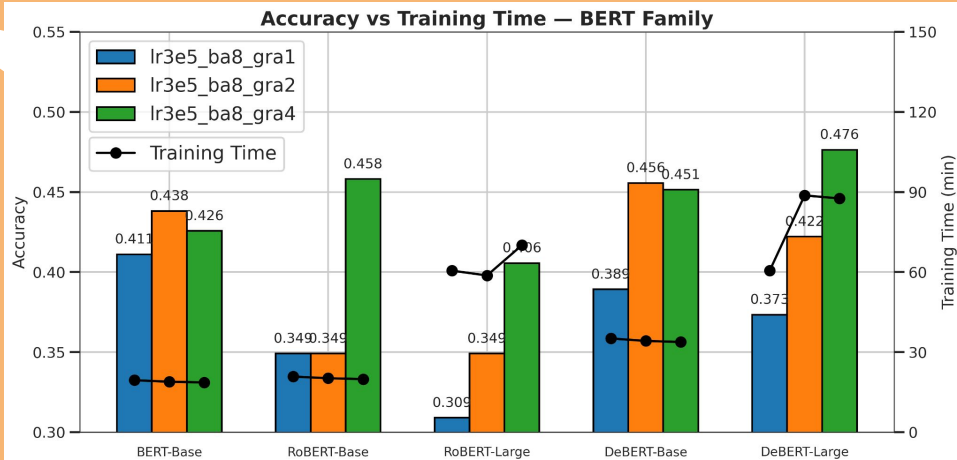
Method	Hidden Layer
MLP	Input → 512 → Output
MLP	Input → 512 → 256 → Output
MLP	Input → 512 → 128 → Output

Exp Results - CNN



Model	Standard CNN Architecture	CNN With Skip Connection
CNN0	Emb → Conv32×3 → Conv32×3 → MP → Conv64×3 → GMP	Emb → [Conv32 → Conv32 + Add] → MP → Conv64 → GMP → Dropout(0.3) or not
CNN1	Emb → Conv32×3 → Conv32×3 → MP → Conv64×3 → Conv64×3 → GMP	Emb → [Conv32 → Conv32 + Add] → MP → [Conv64 → Conv64 + Add] → GMP → Dropout(0.3) or not
CNN2	Emb → Conv32×3 → Conv32×3 → MP → Conv64×3 → Conv64×3 → MP → Conv128×3 → GMP	Emb → [Conv32 → Conv32 + Add] → MP → [Conv64 → Conv64 + Add] → MP → Conv128 → GMP → Dropout(0.3) or not
CNN3	Emb → Conv32×3 → Conv32×3 → MP → Conv64×3 → Conv64×3 → MP → Conv128×3 → Conv128×3 → GMP	Emb → [Conv32 → Conv32 + Add] → MP → [Conv64 → Conv64 + Add] → MP → [Conv128 → Conv128 + Add] → GMP → Dropout(0.3) or not
CNN4	Emb → Conv32×3 → Conv32×3 → MP → Conv64×3 → Conv64×3 → MP → Conv128×3 → Conv128×3 → MP → Conv256×3 → GMP	Emb → [Conv32 → Conv32 + Add] → MP → [Conv64 → Conv64 + Add] → MP → [Conv128 → Conv128 + Add] → MP → Conv256 → GMP → Dropout(0.3) or not
CNN5	Emb → Conv32×3 → Conv32×3 → MP → Conv64×3 → Conv64×3 → MP → Conv128×3 → Conv128×3 → MP → Conv256×3 → Conv256×3 → GMP	Emb → [Conv32 → Conv32 + Add] → MP → [Conv64 → Conv64 + Add] → MP → [Conv128 → Conv128 + Add] → MP → [Conv256 → Conv256 + Add] → GMP → Dropout(0.3) or not

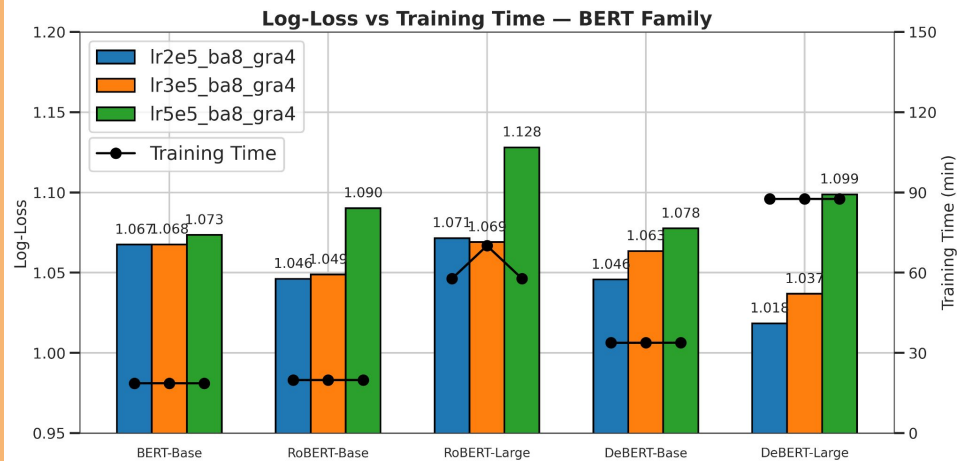
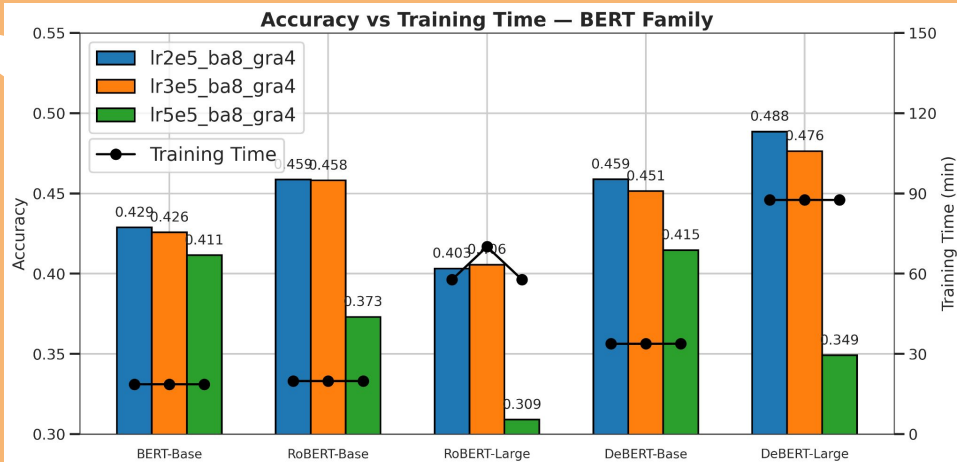
Exp Results - BERT



Model	Developer	Layers	Attention Heads	Parameters
BERT-Base	Google AI	12	12	110M
RoBERTa-Base	Facebook AI (Meta)	12	12	125M
RoBERTa-Large	Facebook AI (Meta)	24	16	355M
DeBERTa-Base	Microsoft Research	12	12	140M
DeBERTa-Large	Microsoft Research	24	16	390M

learning_rate	batch_size	gradient_accumulation_steps
3e-5	8	1
3e-5	8	2
3e-5	8	4

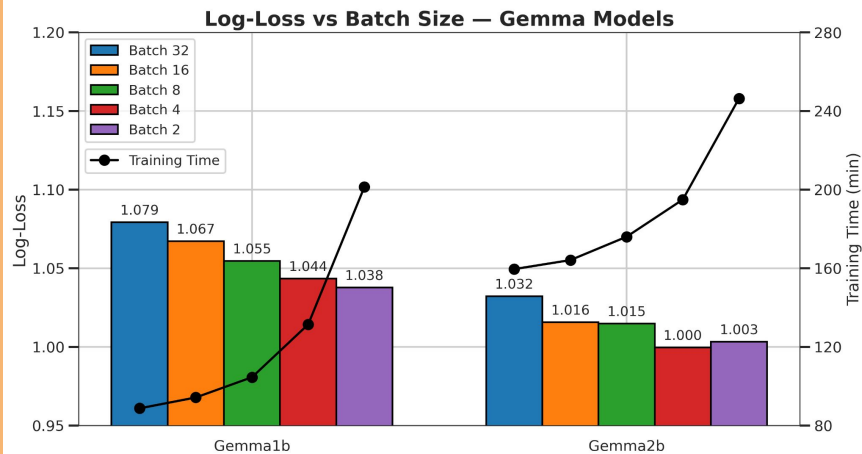
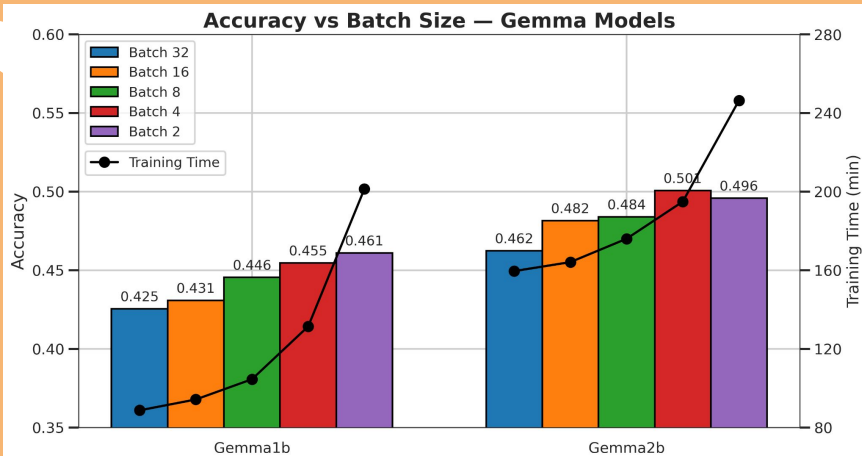
Exp Results - BERT



Model	Developer	Layers	Attention Heads	Parameters
BERT-Base	Google AI	12	12	110M
RoBERT-Base	Facebook AI (Meta)	12	12	125M
RoBERT-Large	Facebook AI (Meta)	24	16	355M
DeBERT-Base	Microsoft Research	12	12	140M
DeBERT-Large	Microsoft Research	24	16	390M

learning_rate	batch_size	gradient_accumulation_steps
2e-5	8	4
3e-5	8	4
5e-5	8	4

Exp Results - Gemma



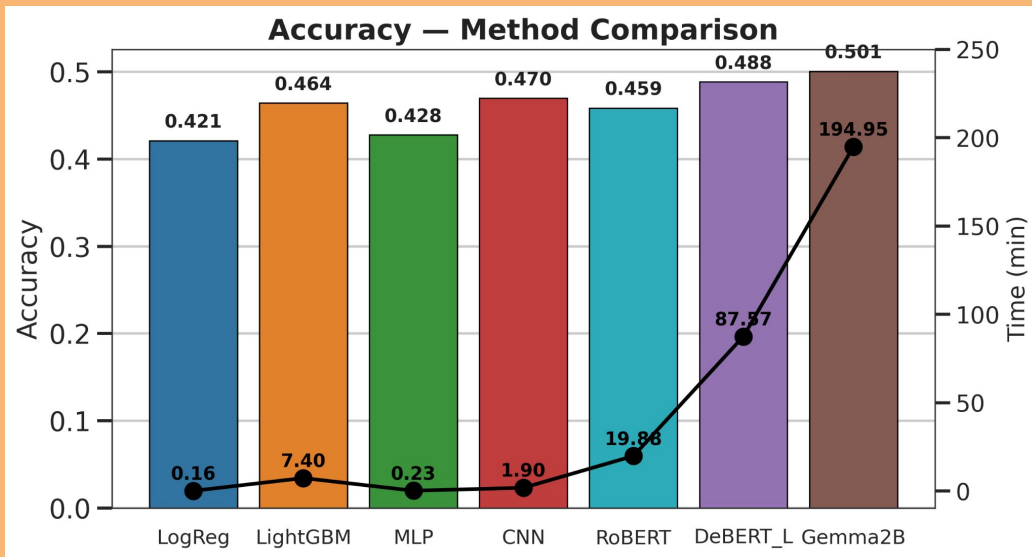
Model	Developer	Parameters	Layers	Attention Heads
Gemma-1B	Google DeepMind	1.04B	24	16
Gemma-2B	Google DeepMind	2.06B	26	16

batch_size
32
16
8
4
5

Exp Results

Accuracy: How many total predictions were correct

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

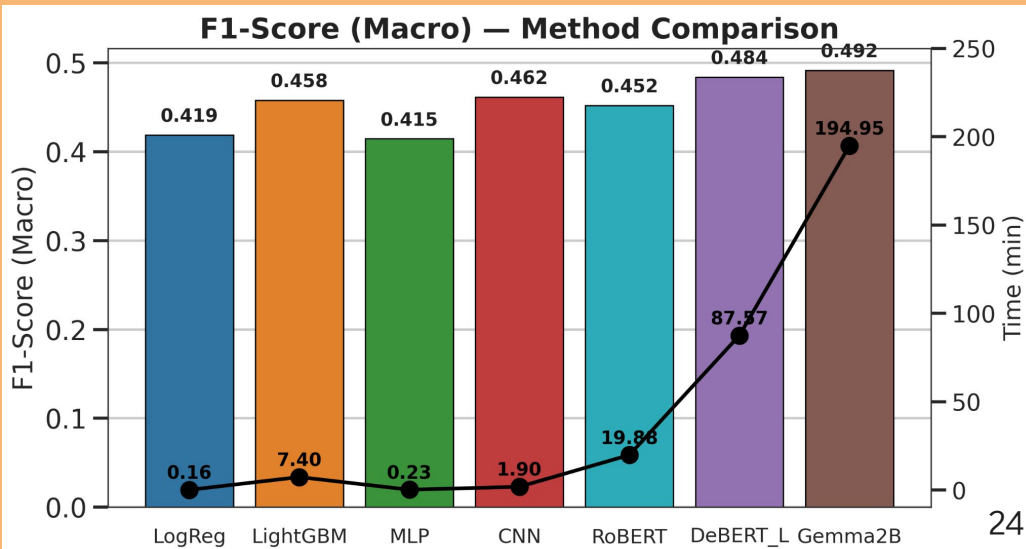
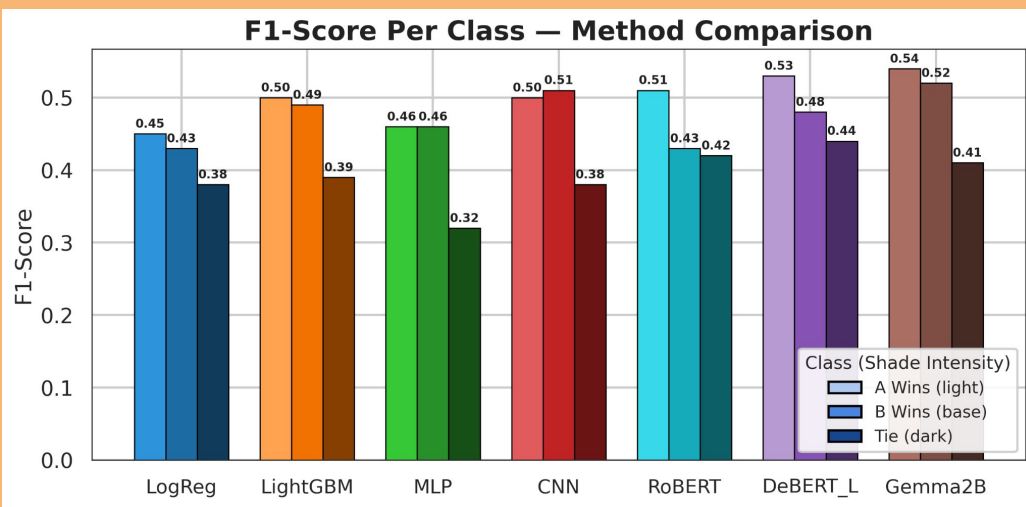


Exp Results

F1-Score (Macro Average):

Measures the balance between precision and recall across all classes by computing each class's F1-score (A-wins, B-wins, Tie) and averaging them equally.

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

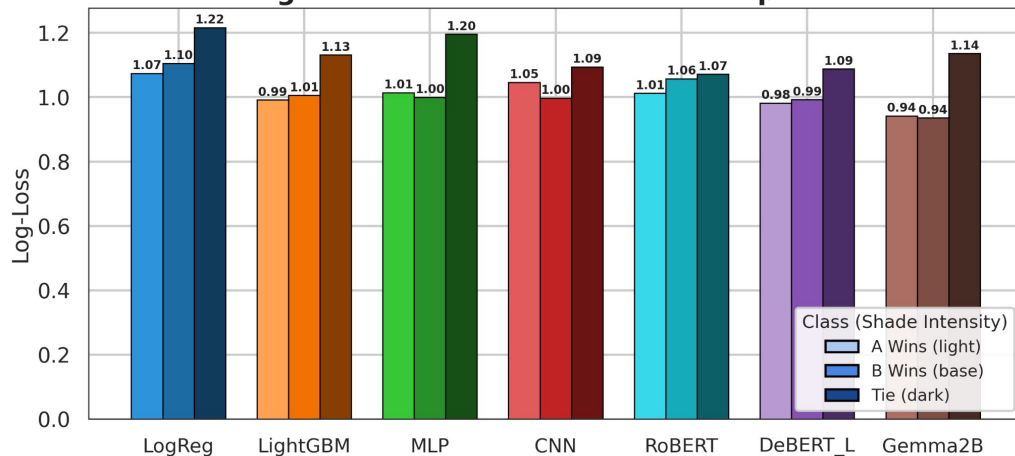


Exp Results

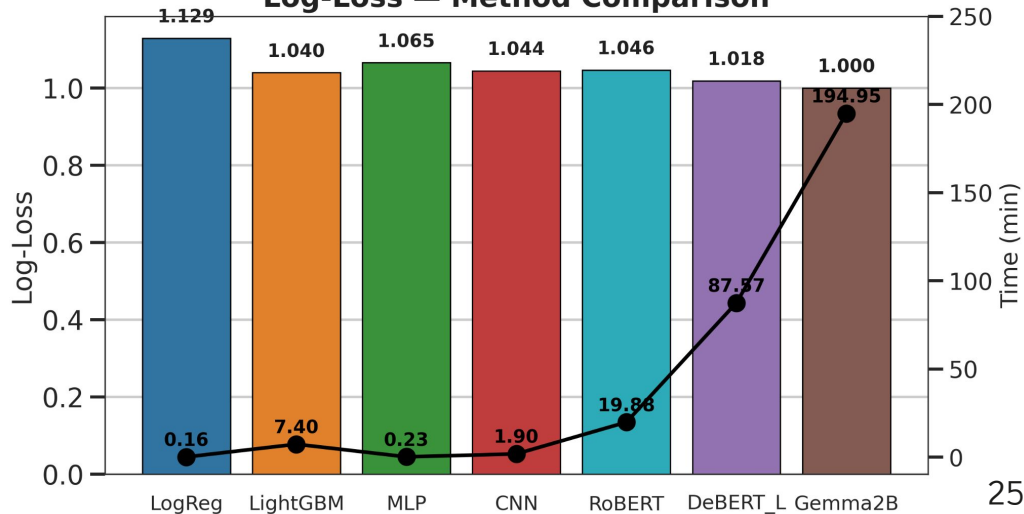
Log-Loss (cross-entropy loss)
measures how well a model's
predicted probabilities align with
the actual class labels.

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Log-Loss Per Class — Method Comparison



Log-Loss — Method Comparison

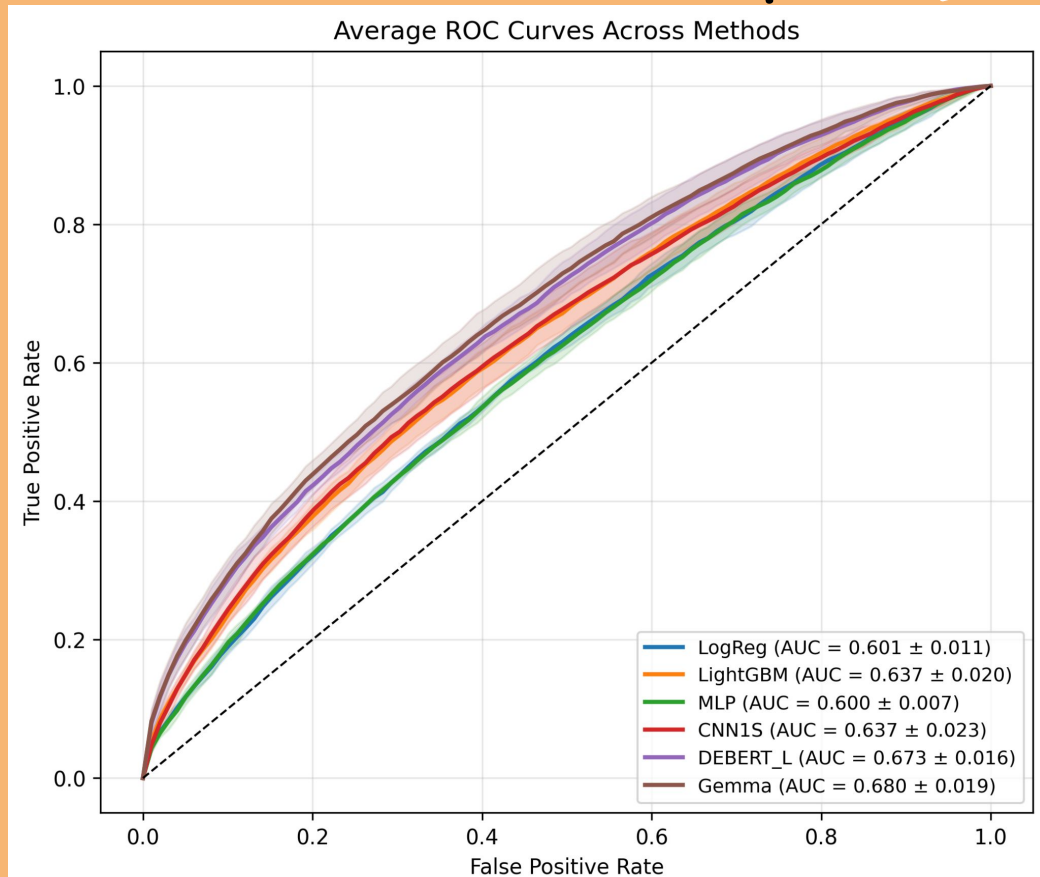


Exp Results

ROC Curve shows how well a model distinguishes between classes by plotting **True Positive Rate** vs. **False Positive Rate**.

A higher curve and larger AUC indicate better discrimination and overall model performance.

$$AUC = \int_0^1 TPR(FPR)d(FPR)$$



Exp Results

Performance of Test

Official test.csv is unlabeled.

Confidence patterns — some models produce sharper probabilities, others are more uncertain.

Class tendencies — certain models favor one class (e.g., BERT often predicts class 1).

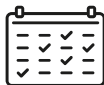
Methods	id	prediction	proba_class_0	proba_class_1	proba_class_2
LogReg	136060	1	0.07	0.48	0.45
	211333	0	0.60	0.23	0.18
	1233961	0	0.40	0.38	0.22
LightGBM	136060	2	0.19	0.29	0.52
	211333	2	0.35	0.25	0.40
	1233961	1	0.28	0.45	0.27
MLP	136060	1	0.19	0.59	0.22
	211333	2	0.38	0.23	0.40
	1233961	1	0.35	0.44	0.21
CNN	136060	2	0.23	0.32	0.45
	211333	0	0.38	0.27	0.34
	1233961	1	0.25	0.49	0.26
BERT	136060	1	0.16	0.73	0.11
	211333	1	0.13	0.61	0.26
	1233961	1	0.31	0.40	0.29
Gemma	136060	2	0.15	0.27	0.58
	211333	1	0.19	0.50	0.31
	1233961	0	0.41	0.27	0.32

Future Work



Hyperparameter Interaction

Explore how multiple hyperparameters jointly affect model performance.



Real-World Implementation

Deploy models with real user interaction to validate offline results.



Performance Optimization

Use insights from interaction and experiments to fine-tune models for speed, accuracy, and usability.



Reference

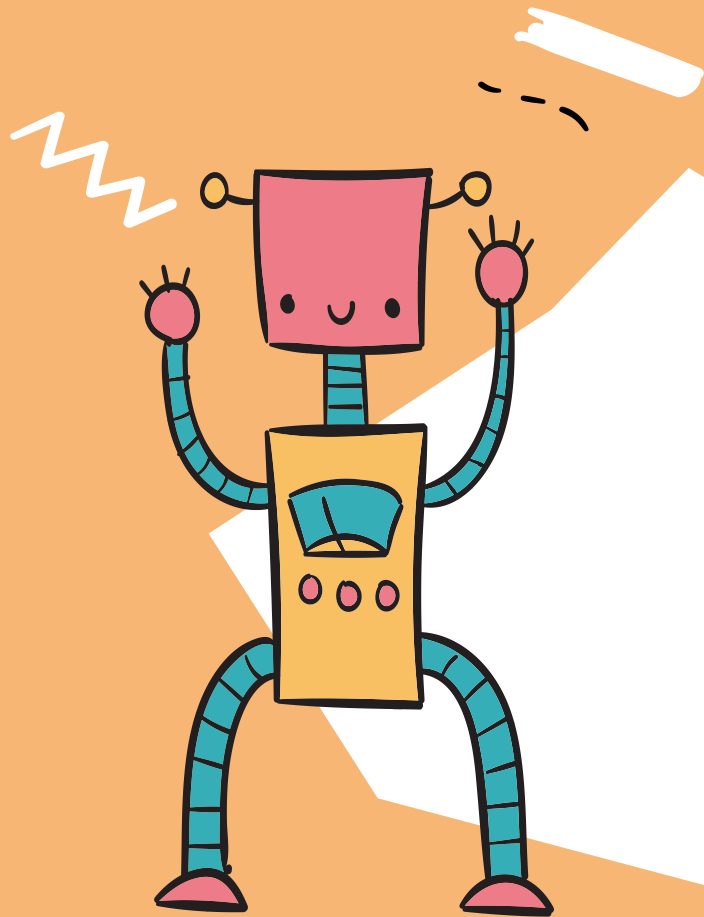
- [1] Gao, Ge, et al. “Aligning LLM Agents by Learning Latent Preference from User Edits.” Advances in Neural Information Processing Systems, vol. 37, 2024, pp. 136873–136896.
- [2] Qiu, Zhangchi, et al. “Unveiling User Preferences: A Knowledge Graph and LLM-Driven Approach for Conversational Recommendation.” arXiv preprint arXiv:2411.14459, 2024.
- [3] Austin, David, et al. “Bayesian Optimization with LLM-Based Acquisition Functions for Natural Language Preference Elicitation.” Proceedings of the 18th ACM Conference on Recommender Systems, 2024.
- [4] Zhao, Yuyue, et al. “Let Me Do It for You: Towards LLM Empowered Recommendation via Tool Learning.” Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2024.
- [5] Zhang, Zijian, et al. “LLM-Powered User Simulator for Recommender System.” Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, no. 12, 2025.
- [6] Wei-lin Chiang, Lianmin Zheng, Lisa Dunlap, Joseph E. Gonzalez, Ion Stoica, Paul Mooney, Sohier Dane, Addison Howard, and Nate Keating. LLM Classification Finetuning. <https://kaggle.com/competitions/llm-classification-finetuning>, 2024. Kaggle.
- [7] Chajia, M., & Nfaoui, E. H. (2024). Customer Churn Prediction Approach Based on LLM Embeddings and Logistic Regression. Future Internet, 16(12), 453. <https://doi.org/10.3390/fi16120453>
- [8] Jian-Bo Yang, et al. “Feature Selection for MLP Neural Network: The Use of Random Permutation of Probabilistic Outputs.” IEEE Transactions on Neural Networks, vol. 20, no. 12, Dec. 2009, pp. 1911–1922, <https://doi.org/10.1109/tnn.2009.2032543>.
- [9] Devlin, Jacob, et al. “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding.” ArXiv, 11 Oct. 2018, arxiv.org/abs/1810.04805.

+

Thanks!

Does anyone have any questions?

CREDITS: This presentation template was created by [Slidesgo](#), including icons by [Flaticon](#), infographics & images by [Freepik](#)

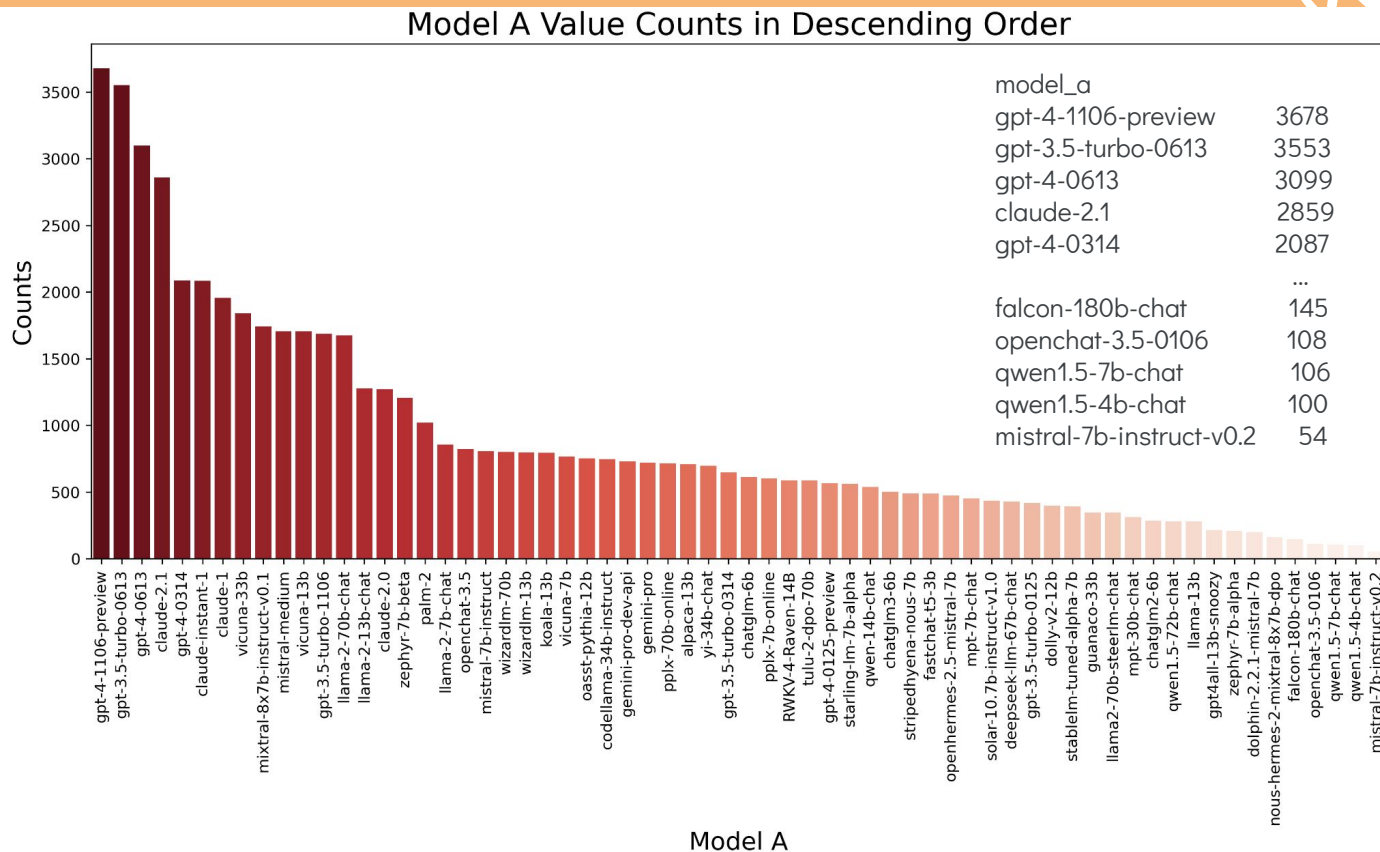


+

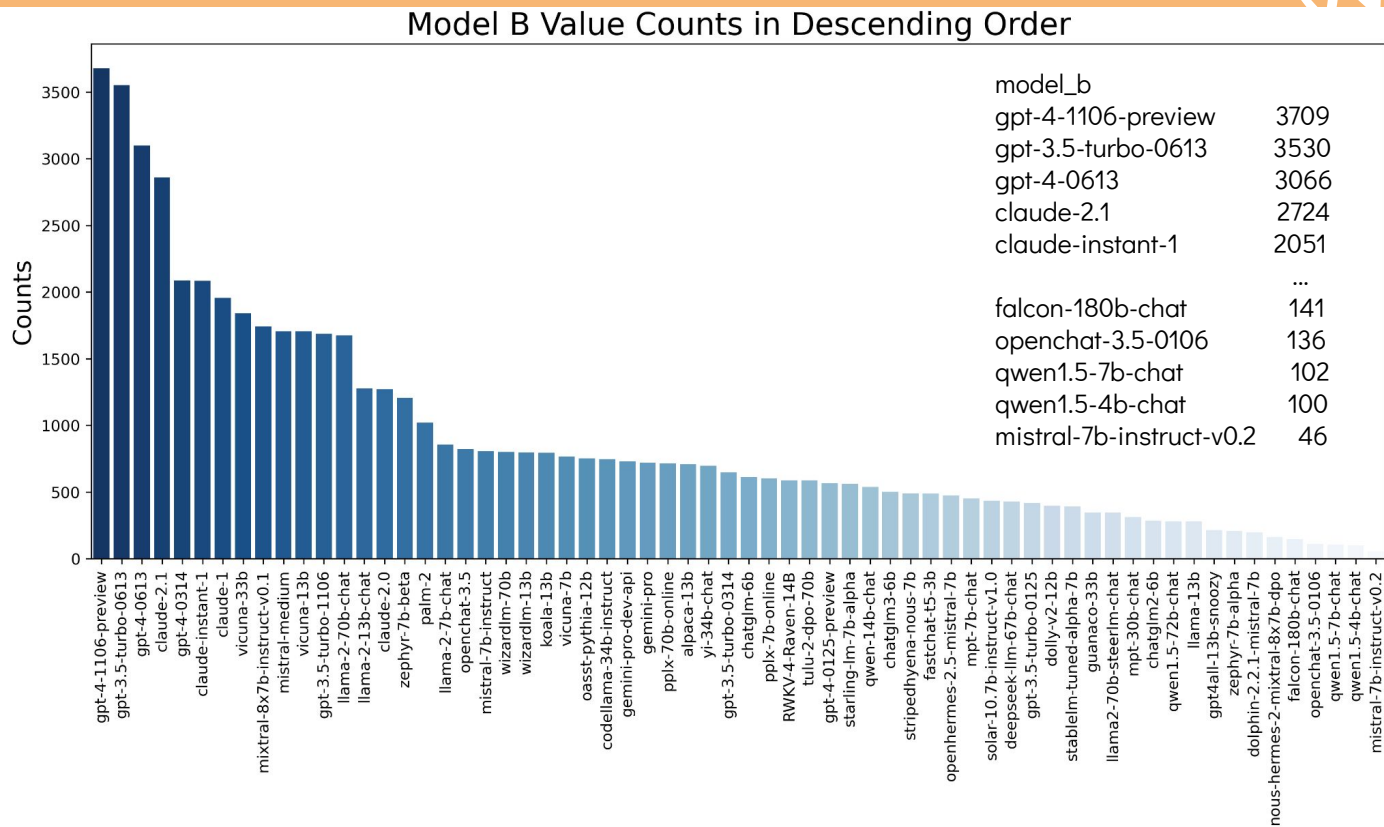
Appendix – Dataset Distribution

	id	model_a	model_b	prompt	response_a	response_b	winner_model_a	winner_model_b	winner_tie
0	30192	gpt-4-1106-preview	gpt-4-0613	["Is it morally right to try to have a certain...	["The question of whether it is morally right ...	["As an AI, I don't have personal beliefs or o...	1	0	0
1	53567	koala-13b	gpt-4-0613	["What is the difference between marriage lice...	["A marriage license is a legal document that ...	["A marriage license and a marriage certificat...	0	1	0
2	65089	gpt-3.5-turbo-0613	mistral-medium	["explain function calling. how would you call...	["Function calling is the process of invoking ...	["Function calling is the process of invoking ...	0	0	1
3	96401	llama-2-13b-chat	mistral-7b-instruct	["How can I create a test set for a very rare ...	["Creating a test set for a very rare category...	["When building a classifier for a very rare c...	1	0	0
4	198779	koala-13b	gpt-3.5-turbo-0314	["What is the best way to travel from Tel-Aviv...	["The best way to travel from Tel Aviv to Jeru...	["The best way to travel from Tel-Aviv to Jeru...	0	1	0

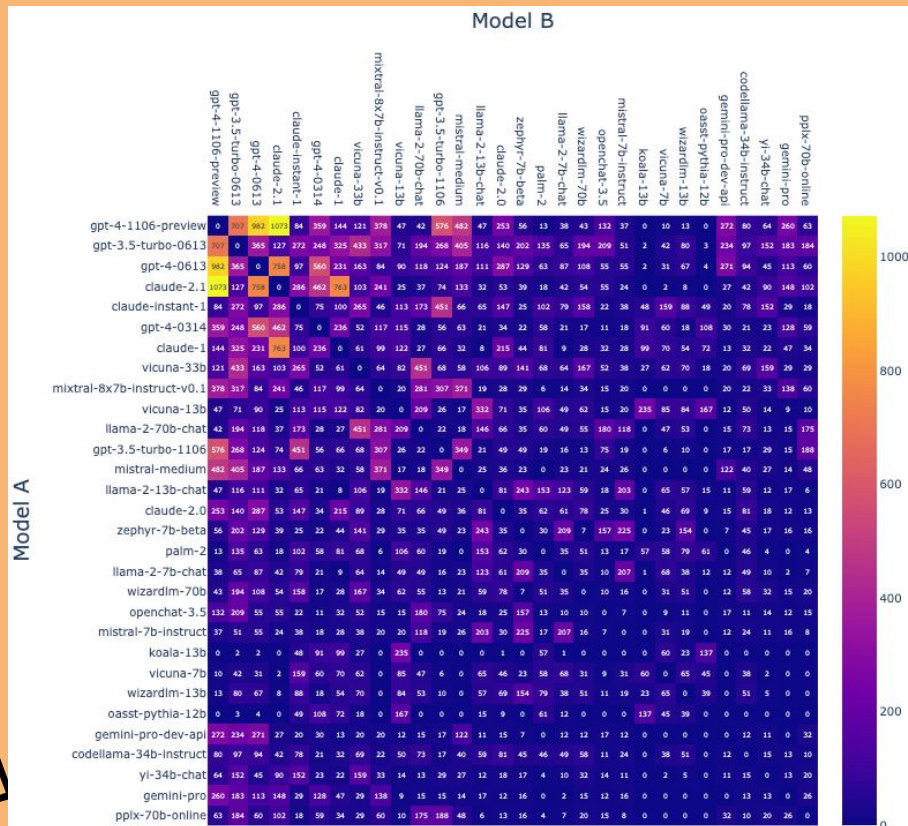
Appendix – Dataset Distribution



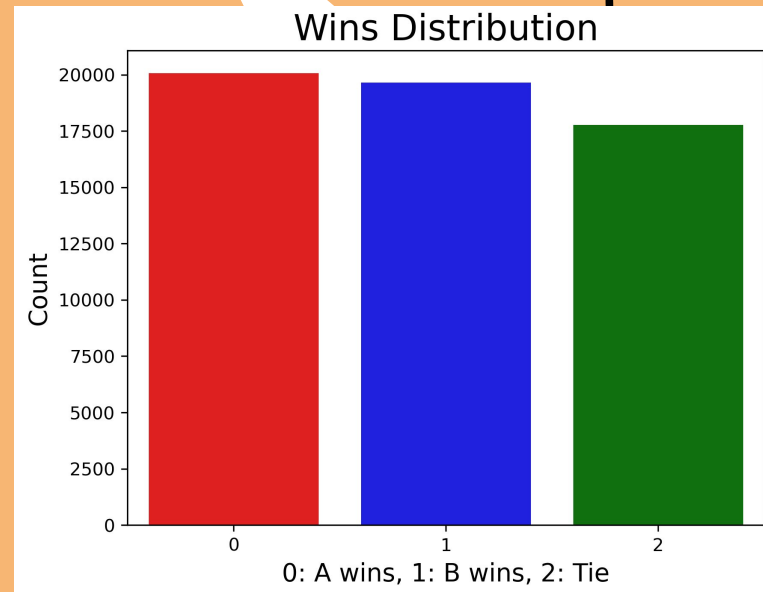
Appendix – Dataset Distribution



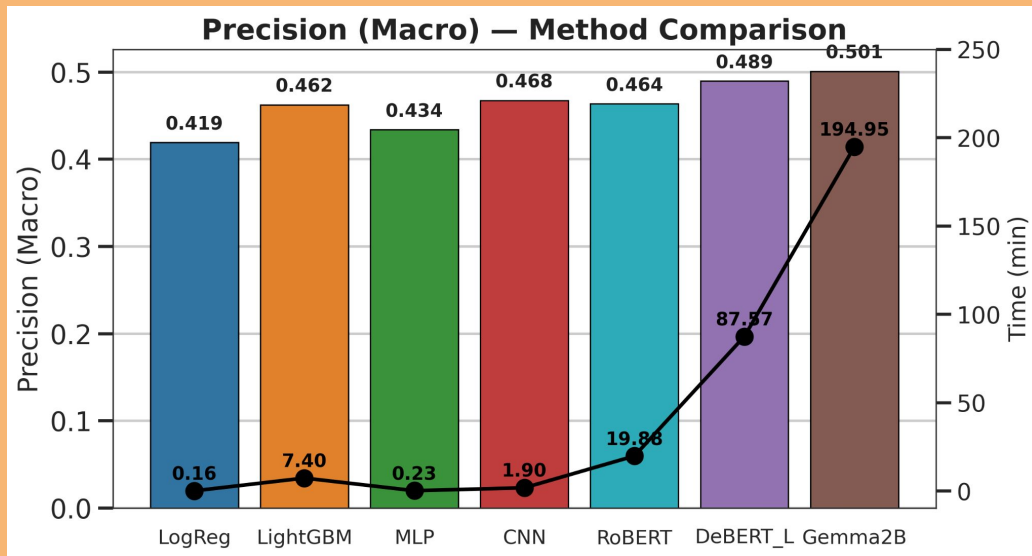
- Dataset Distribution



Battle Count of Each Combination of Models



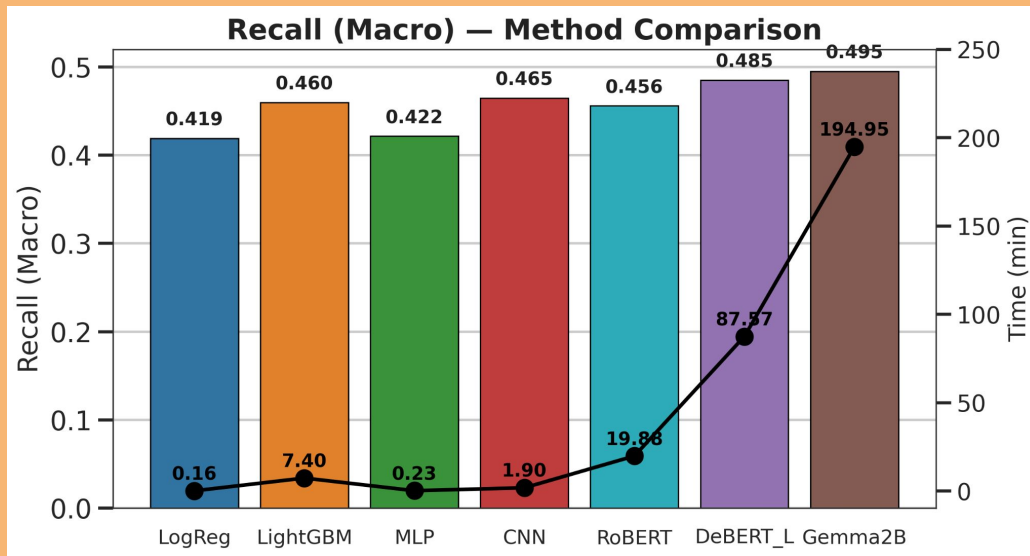
Appendix – Precision



Precision: how many predicted positives were correct.

$$Precision = \frac{TP}{TP + FP}$$

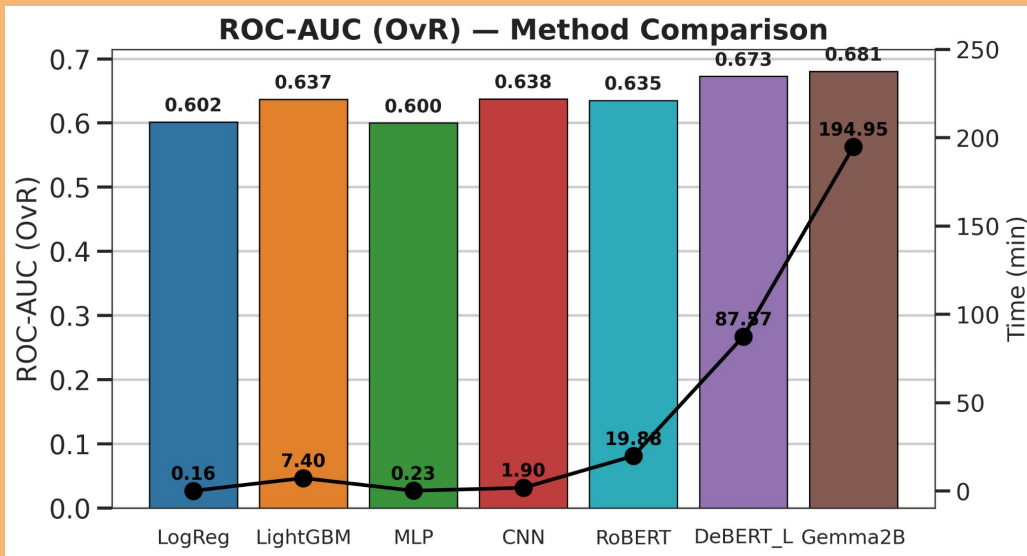
Appendix – Recall



Recall: how many actual positives were caught.

$$Recall = \frac{TP}{TP + FN}$$

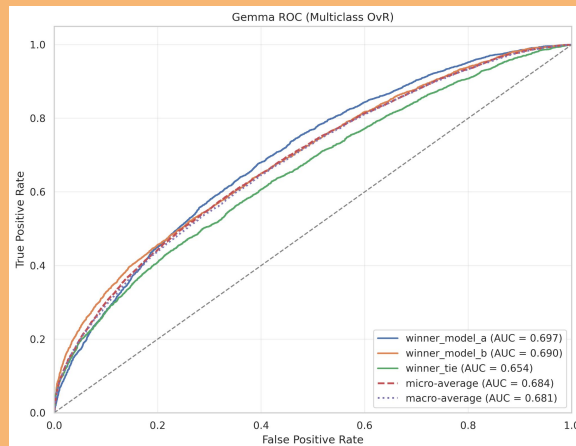
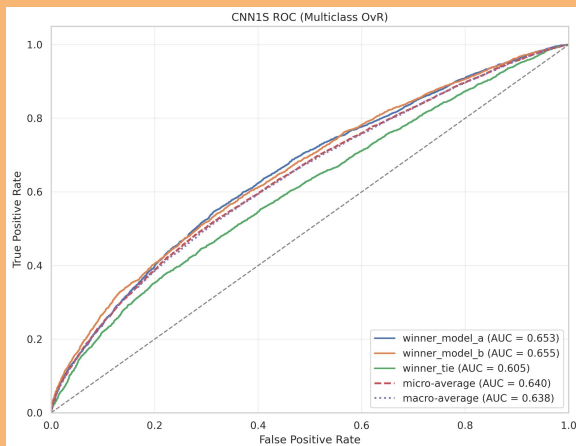
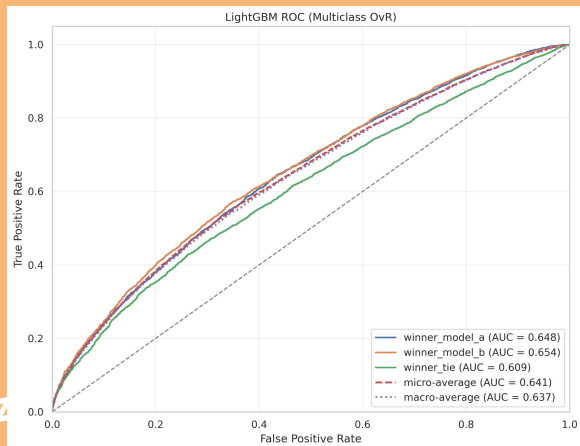
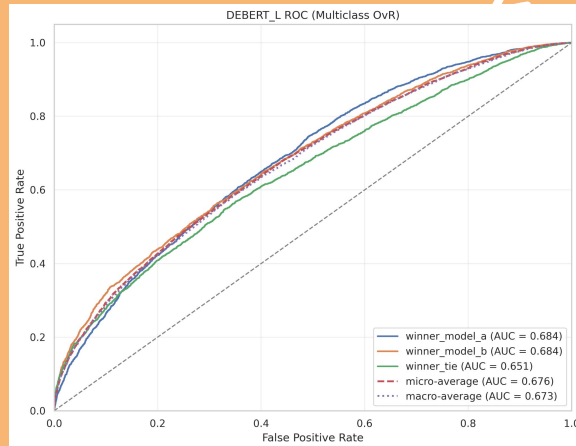
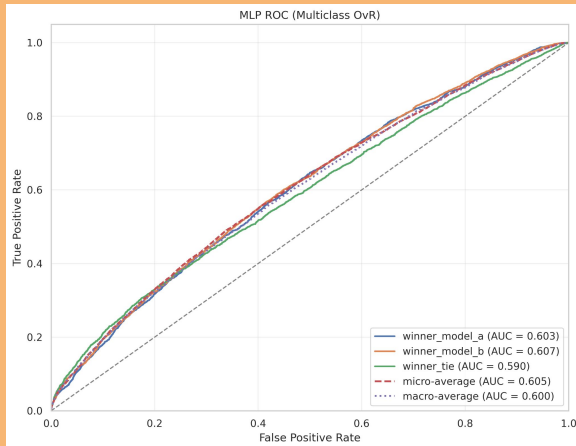
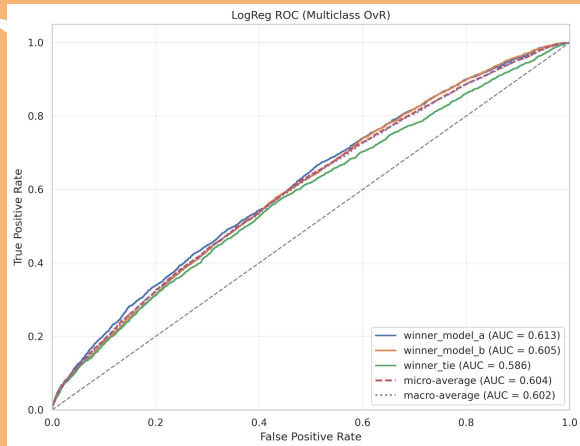
Appendix – ROC-AUC (OvR)



ROC-AUG using One-vs-Rest (OvR) multiclass strategy. For each class, the model treats that class as “positive” and all others as “negative,” calculates its AUC,

$$LogLoss = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Appendix – ROC Curve (per-class)



Appendix - Confusion Matrix

